Project Title

Delivery Route Optimizer

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Immella Vikram (2520090118)
Challa Goutham Teja (2520090225)

Team Number: 29

Section: 11

Under the guidance of

Mrs. Ch. Anitha

Asst. Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled **"Delivery Route Optimizer"** is a record of Bonafide work of **Immella Vikram – 2520090118, Challa Goutham Teja – 2520090225** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Immella Vikram – 2520090118

Challa Goutham Teja – 2520090225

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **"Delivery Route Optimizer"** is a bonafide work done and submitted by **Immella Vikram – 2520090118, Challa Goutham Teja – 2520090225** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our guide, Mrs. Ch. Anitha, Asst. Professor, Department of Computer Science and Information Technology, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted us directly or indirectly for successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Immella Vikram – 2520090118

Challa Goutham Teja – 2520090225

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	15
8.	Conclusion & Future Scope	20
9.	References	21

1.ABSTRACT

The Delivery Route Optimizer is an efficient solution designed to find the best possible routes for delivering packages to multiple locations while minimizing travel distance and delivery time. In modern delivery and logistics systems, selecting an efficient route is important for reducing fuel consumption, transportation costs, and delays.

This project represents delivery locations as nodes and the roads connecting them as edges in a graph. The system uses suitable data structures and graph algorithms to analyse different routes and identify the most efficient path between locations. Dijkstra's algorithm is used to find the shortest path based on distance or travel cost, while data structures such as graphs, adjacency lists, and priority queues help in storing and processing route information efficiently.

The system can handle multiple delivery locations and determine an optimized sequence of routes for the delivery vehicle. It is implemented as a Java console program and as a web application with an animated route map. By reducing unnecessary travel and selecting shorter paths, the proposed solution improves delivery efficiency and saves time and resources, demonstrating the practical application of data structures and graph algorithms to real-world transportation and logistics problems.

Team Name: (Team Number: 29)

Team Members:

1. Immella Vikram, 2520090118
2. Challa Goutham Teja, 2520090225

2. INTRODUCTION

Efficient package delivery is a core challenge in modern logistics. Courier companies, online shops and food suppliers must reach many locations every day, and a poorly chosen route wastes fuel, time and money. Deciding which stop to visit next, and which roads to take between stops, therefore has a direct effect on cost and on customer satisfaction.

This project, the Delivery Route Optimizer, models the problem with graph theory. Each place is a node and each road is a weighted edge that holds its length in kilometres. The graph is stored as an adjacency list, which suits road networks because most places connect to only a few others. Dijkstra's shortest-path algorithm, supported by a priority queue built as a min-heap, finds the shortest path from the depot to every delivery point. The van then goes to the nearest remaining stop each time, and the result is compared with a simple A, B, C order to show how much distance is saved.

The system is built in two forms: a Java console program that prints distances, paths and the optimized sequence, and a browser-based web application with Dashboard, Route Optimizer, Deliveries, History and Settings pages. In the web application the user can select stops, click on the map to add new stops, choose whether the van returns to the depot, and watch an animated van complete the deliveries. The project links directly to the DSA-3 course themes of graph algorithms and priority-queue-based optimisation.

The rest of this report is organised as follows: Section 3 reviews related work, Section 4 states the problem and objectives, Section 5 lists the hardware and software requirements, Section 6 presents the main code modules, Section 7 shows the results, Section 8 gives the conclusion and future scope, and Section 9 lists the references.

3. LITERATURE SURVEY

Dijkstra's algorithm [1], [3]. Proposed by E. W. Dijkstra in 1959, it finds the shortest path from one source node to every other node in a graph whose edge weights are not negative. It repeatedly picks the unvisited node with the smallest known distance and relaxes its outgoing edges. With a binary min-heap it runs in O((V + E) log V) time. It is the classical base of most route-planning systems and the core of this project.

Bellman–Ford algorithm [4]. Bellman–Ford also computes shortest paths from one source, but it works even when some edge weights are negative and it can detect negative cycles. It relaxes every edge V − 1 times, so it runs in O(V·E) time, which is slower than Dijkstra's algorithm. Road lengths are never negative, so Dijkstra's algorithm is the better choice for delivery routes.

A* search [5]. A* extends Dijkstra's algorithm with a heuristic estimate, such as the straight-line distance to the goal, so that the search moves towards the destination instead of spreading in all directions. It is widely used in games and navigation software for fast point-to-point routing. Our project needs shortest distances from the depot to many stops, so plain Dijkstra fits it better.

Travelling Salesman Problem and the nearest-neighbour heuristic [6]. TSP asks for the shortest tour that visits every place once. It is NP-hard, so large cases are solved with heuristics. The nearest-neighbour heuristic always goes to the closest unvisited place; it is simple and fast but does not always give the best tour. Our project uses this idea, with Dijkstra's shortest path as the distance, to choose the next delivery stop.

Vehicle Routing Problem [7]. VRP generalises TSP to a fleet of vehicles that must serve many customers from a depot, often with capacity and time-window limits. It is the standard model for real delivery planning and is also NP-hard, so practical systems use heuristic and approximation methods. This project solves the single-vehicle case, which is the basic building block of VRP.

Google OR-Tools routing library [8]. OR-Tools is an open-source solver from Google that handles TSP and VRP variants with constraints such as capacities and time windows. It gives good solutions on large problems but hides the inner steps from a learner. Our project builds the graph, the heap and Dijkstra's routine from first principles so that every step can be seen and explained.

OSRM [9] and GraphHopper [10]. OSRM and GraphHopper are open-source routing engines that compute routes on real road networks built from OpenStreetMap data. OSRM uses pre-processing methods such as contraction hierarchies to answer queries quickly on very large maps. These engines work on huge real maps and need a server set-up, while our project uses a small sample map with an animated interface to explain the core algorithm.

Data structure choices. A road network is sparse, so an adjacency list uses less memory than an adjacency matrix and lets Dijkstra's algorithm visit only the real neighbours of a place. A priority queue (min-heap) gives the closest unvisited place in O(log V) time. Both choices are used in this project.

Gap addressed. Most existing tools are large libraries or web services. This project gives a clean, from-first-principles implementation of the graph + Dijkstra pipeline, in Java and as a web application, to show clearly the algorithmic ideas behind real-world routing systems.

4. Problem Statement and Objectives

Problem Statement:

Given a set of delivery locations connected by roads with distances, determine the most efficient route for a delivery vehicle to reach the required locations from a central depot, minimizing total travel distance and delivery time.

Problem Description:

Visiting stops in an arbitrary order wastes distance, time and fuel. A method is needed that finds the shortest paths between places and a good order in which to visit the stops.

Objectives:

- Represent the delivery network as a weighted graph using adjacency lists.
- Implement Dijkstra's algorithm to compute shortest paths from the depot to all delivery points.
- Use a priority queue (min-heap) to select the next closest unvisited location efficiently.
- Determine an optimized delivery sequence covering multiple locations using the nearest remaining stop.
- Compare the optimized route with a naive (A, B, C) order to show the distance saved.
- Provide a simple interface, a Java console program and a web application, to enter locations and roads and view the computed route.
- Let the user add new stops on the map, track deliveries and keep a history of the routes planned.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Standard PC / laptop
- Dual-core processor or higher
- Minimum 4 GB RAM (8 GB recommended)
- Minimum 500 MB free disk space
- Display of 1366 × 768 resolution or higher (for the web application)

Software Requirements:

- Programming languages: Java (console program) and HTML, CSS, JavaScript (web application)
- Java Development Kit (JDK 8 or later) to compile and run the console program
- Web browser: latest Google Chrome, Microsoft Edge or Mozilla Firefox
- IDE / editor: VS Code, Eclipse or IntelliJ IDEA
- Operating system: Windows, Linux or macOS
- Git (optional, for version control)

6. IMPLEMENTATION

The project has two implementations: a Java console program (RouteOptimizer.java) and a web application (index.html) that uses the same graph, min-heap and Dijkstra logic. The four main modules are given below.

Module 1: Min-Heap priority queue (RouteOptimizer.java)

A min-heap built from scratch. push() adds an entry and moves it up; pop() removes the smallest distance and moves the last entry down.

```
static class MinHeap {
    private int[][] heap = new int[16][];   // each entry: {distance, node}
    private int size = 0;

    boolean isEmpty() { return size == 0; }

    void push(int dist, int node) {
        if (size == heap.length) heap = Arrays.copyOf(heap, size * 2);
        heap[size] = new int[]{dist, node};
        int i = size++;
        while (i > 0) {                       // sift up
            int parent = (i - 1) / 2;
            if (heap[parent][0] <= heap[i][0]) break;
            swap(i, parent);
            i = parent;
        }
    }

    int[] pop() {
        int[] top = heap[0];
        heap[0] = heap[--size];
        int i = 0;
        while (true) {                        // sift down
            int l = 2 * i + 1, r = l + 1, min = i;
            if (l < size && heap[l][0] < heap[min][0]) min = l;
            if (r < size && heap[r][0] < heap[min][0]) min = r;
            if (min == i) break;
            swap(i, min);
            i = min;
        }
        return top;
    }

    private void swap(int a, int b) {
        int[] t = heap[a]; heap[a] = heap[b]; heap[b] = t;
    }
}
```

Module 2: Graph (adjacency list) and Dijkstra's algorithm (RouteOptimizer.java)

Places are nodes and roads are undirected edges stored in adjacency lists. dijkstra() fills dist[] and prev[] and path() rebuilds a route.

```
static class Graph {
    final int n;
    final String[] names;
```

```
    final List<List<Edge>> adj = new ArrayList<>();

    Graph(String[] names) {
        this.n = names.length;
        this.names = names;
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    }

    void addRoad(int a, int b, int km) {      // undirected road
        adj.get(a).add(new Edge(b, km));
        adj.get(b).add(new Edge(a, km));
    }

    // Dijkstra from src. Fills dist[] and prev[].
    void dijkstra(int src, int[] dist, int[] prev) {
        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(prev, -1);
        dist[src] = 0;
        MinHeap pq = new MinHeap();
        pq.push(0, src);
        while (!pq.isEmpty()) {
            int[] top = pq.pop();
            int d = top[0], u = top[1];
            if (d > dist[u]) continue;        // stale entry
            for (Edge e : adj.get(u)) {
                if (d + e.weight < dist[e.to]) {
                    dist[e.to] = d + e.weight;
                    prev[e.to] = u;
                    pq.push(dist[e.to], e.to);
                }
            }
        }
    }

    List<Integer> path(int[] prev, int target) {
        LinkedList<Integer> p = new LinkedList<>();
        for (int v = target; v != -1; v = prev[v]) p.addFirst(v);
        return p;
    }
}
```

Module 3: Delivery sequence planner (RouteOptimizer.java)

optimizedOrder() always goes to the nearest unvisited stop (by shortest-path distance); routeDistance() totals the distance of any order so the two can be compared.

```
// Total distance visiting stops in the given order (depot = 0)
int routeDistance(List<Integer> order, boolean returnToDepot) {
    int cur = 0, total = 0;
    int[] dist = new int[n], prev = new int[n];
    List<Integer> targets = new ArrayList<>(order);
    if (returnToDepot) targets.add(0);
    for (int t : targets) {
        dijkstra(cur, dist, prev);
        if (dist[t] == Integer.MAX_VALUE) return -1;   // unreachable
        total += dist[t];
        cur = t;
    }
    return total;
}

// Greedy: always go to the nearest unvisited stop (by shortest-path distance)
List<Integer> optimizedOrder(List<Integer> stops) {
    List<Integer> remaining = new ArrayList<>(stops), order = new ArrayList<>();
    int[] dist = new int[n], prev = new int[n];
```

```
    int cur = 0;
    while (!remaining.isEmpty()) {
        dijkstra(cur, dist, prev);
        int best = remaining.get(0);
        for (int s : remaining) if (dist[s] < dist[best]) best = s;
        order.add(best);
        remaining.remove(Integer.valueOf(best));
        cur = best;
    }
    return order;
}
```

Module 4: Adding a stop by clicking on the map (index.html, JavaScript)

In the web application a click on an empty spot creates a new stop, joins it to the two nearest places and works out the distances from the map. The listing is reformatted for readability.

```
$('setup').onclick = e => {
  const sv = $('sv'), b = sv.getBoundingClientRect();
  const x = (e.clientX - b.left) * 700 / b.width;   // click position in map units
  const y = (e.clientY - b.top) * 400 / b.height;
  const near = N.findIndex(n => Math.hypot(n.x - x, n.y - y) < 30);
  if (near > 0) {                                   // clicked an existing place:
    stops.has(near) ? stops.delete(near) : stops.add(near);   // select / unselect
    chips(); setup(); return;
  }
  if (near === 0) return;                           // the depot is not a stop
  if (N.length >= 14) {
    $('addmsg').textContent = 'Maximum number of stops reached.'; return;
  }
  const px = Math.max(30, Math.min(670, x)), py = Math.max(30, Math.min(370, y));
  const id = N.length, name = String.fromCharCode(64 + id);   // next letter: F, G, ...
  // join the new stop to its 2 nearest places; 1 km = 45 map units (minimum 1 km)
  const ds = N.map((n, i) => {
      const d = Math.hypot(n.x - px, n.y - py);
      return [Math.max(1, Math.round(d / 45)), i, d];
    }).sort((a, b) => a[2] - b[2]).slice(0, 2);
  N.push({ n: name, x: px, y: py });
  ds.forEach(([km, i]) => E.push([i, id, km, 50]));  // road = [from, to, km, speed]
  stops.add(id);
  rebuild(); chips(); setup();
  res = null; $('out').hidden = true; cancelAnimationFrame(raf);
};
```

7. RESULTS & DISCUSSION

```
$ javac RouteOptimizer.java && java RouteOptimizer
=== Delivery Route Optimizer (Dijkstra + Min-Heap) ===
1. Run sample demo
2. Enter my own network
Choose: 1
Shortest distances from depot (Depot):
Delivery PointDistance    Path Used
A             4 km        Depot -> A
B             9 km        Depot -> A -> B
C             7 km        Depot -> C
D             12 km       Depot -> C -> D
E             15 km       Depot -> A -> B -> E

Optimized sequence: Depot -> A -> B -> E -> D -> C  (24 km)
Naive sequence    : Depot -> A -> B -> C -> D -> E  (26 km)
Distance saved    : 2 km (8%)
```

Figure 7.1: Java console output for the sample network

The console program prints the shortest distance and path from the Depot to every delivery point using Dijkstra's algorithm. It then compares the optimized sequence (24 km) with the naive sequence (26 km), saving 2 km (8%).

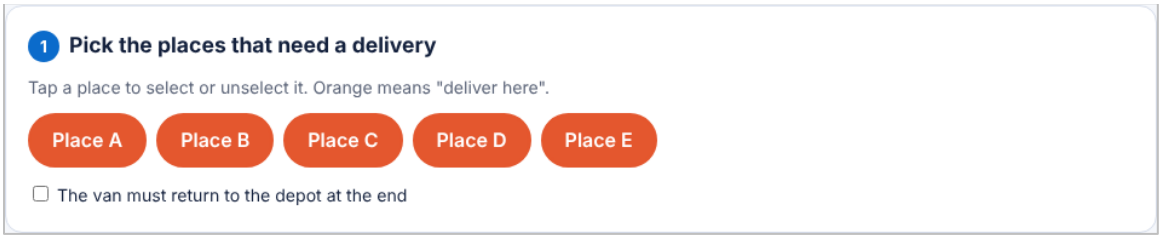


Figure 7.2: Route Optimizer – step 1: choosing the delivery places

Each place is a button; orange means "deliver here". The user can also tick that the van must return to the depot at the end.

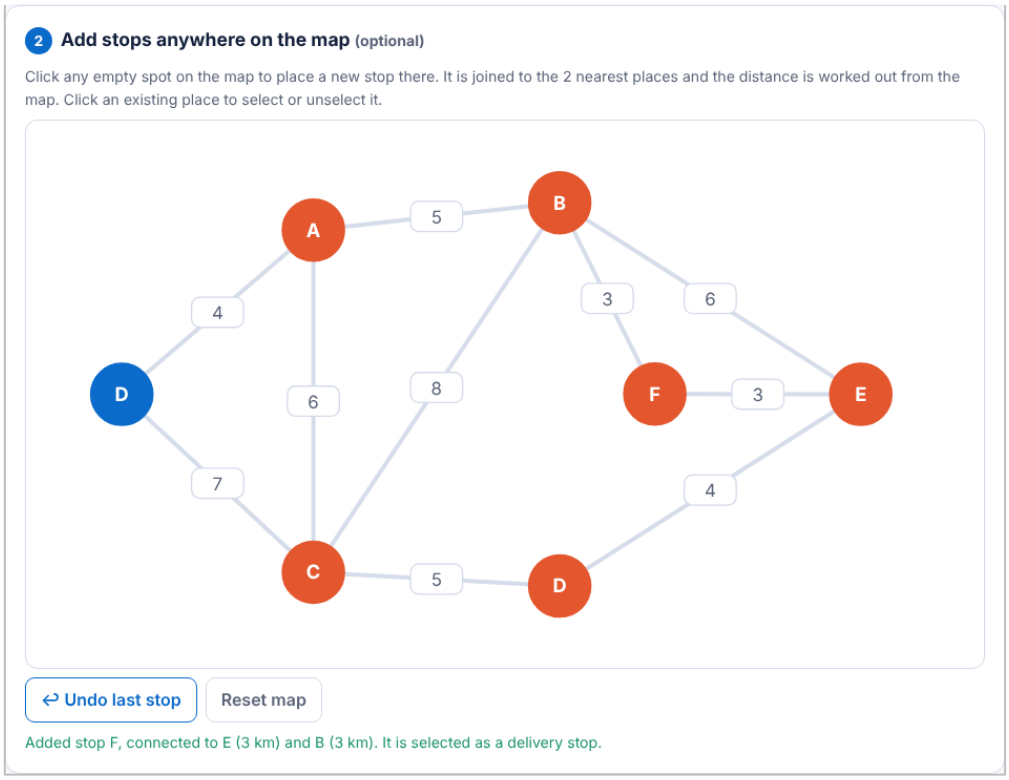


Figure 7.3: Route Optimizer – step 2: adding a stop on the map

Clicking an empty spot on the map adds a new stop (F) joined to its two nearest places, with the distances worked out from the map. Undo last stop and Reset map are available.

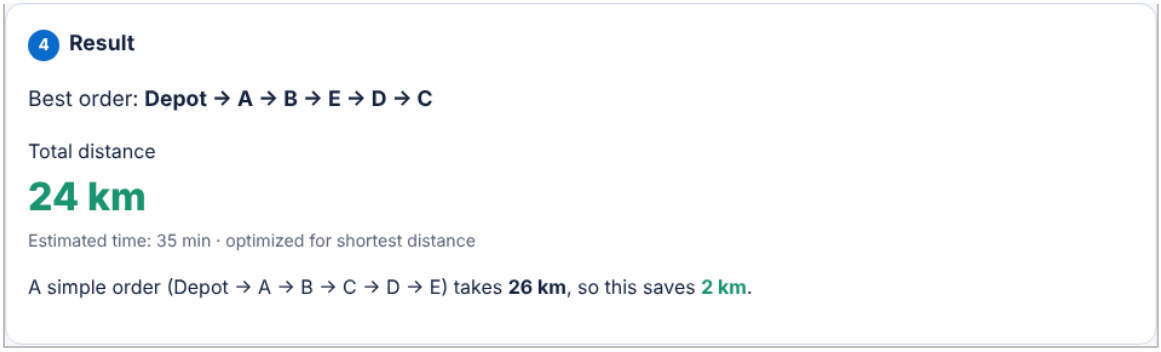


Figure 7.4: Result for the sample map

The best order Depot → A → B → E → D → C is 24 km (35 min). The simple A, B, C order takes 26 km, so the optimized route saves 2 km.

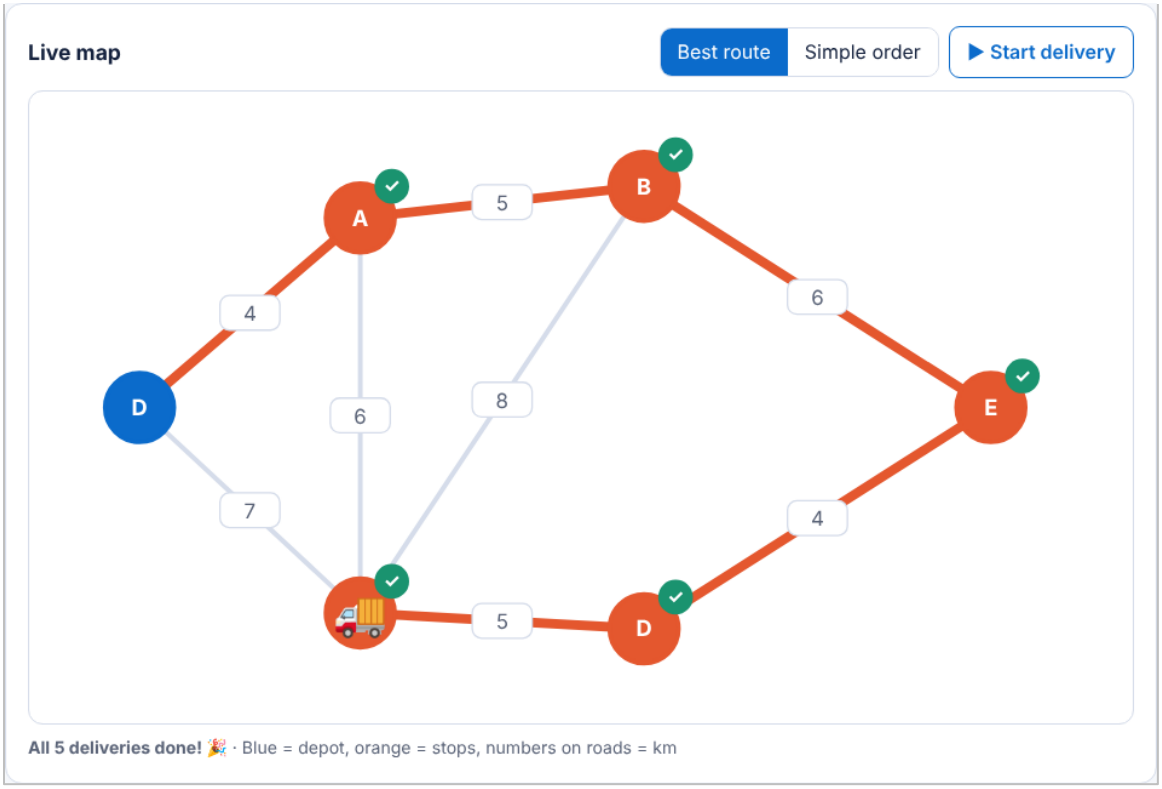


Figure 7.5: Live map after the delivery is complete

The van follows the orange route and each stop turns green with a tick when it is delivered. The Simple order button shows the comparison route on the same map.

Trip, leg by leg

From → To	Distance	Time	Roads taken
Depot → A	4 km	8 min	Depot → A
A → B	5 km	6 min	A → B
B → E	6 km	9 min	B → E
E → D	4 km	8 min	E → D
D → C	5 km	4 min	D → C

Figure 7.6: Trip, leg by leg

Every leg of the trip is listed with its distance, time and the roads taken, so the path between two stops is clear.



Figure 7.7: Result with an added stop and return to the depot

After adding stop F and choosing to return to the depot, the best route is 31 km (39 min) against 41 km for the simple order, saving 10 km.

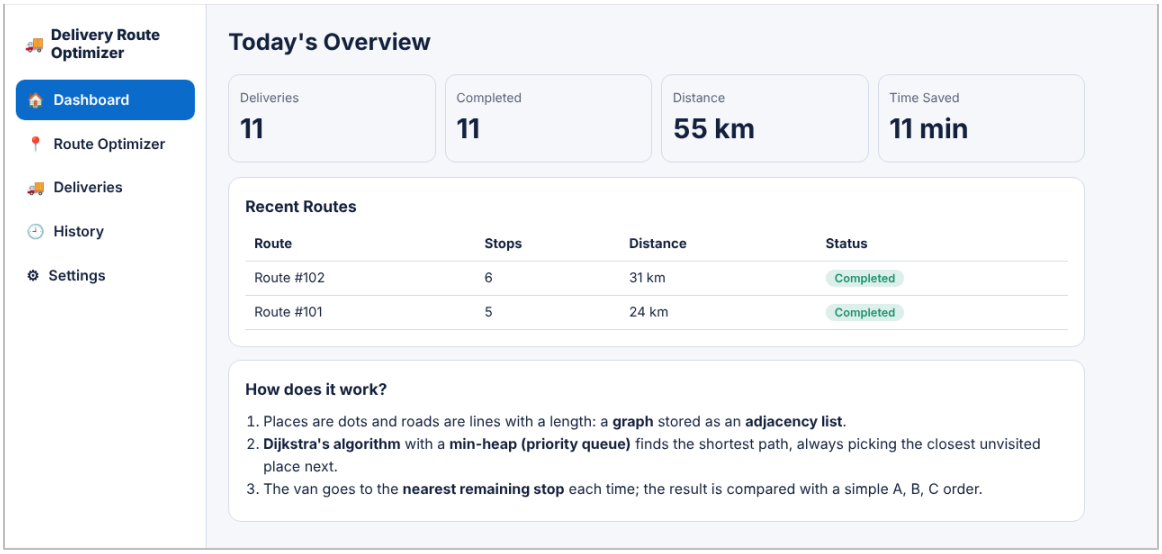


Figure 7.8: Dashboard

Today's overview shows the number of deliveries, completed deliveries, distance and time saved, with the recent routes and their status.

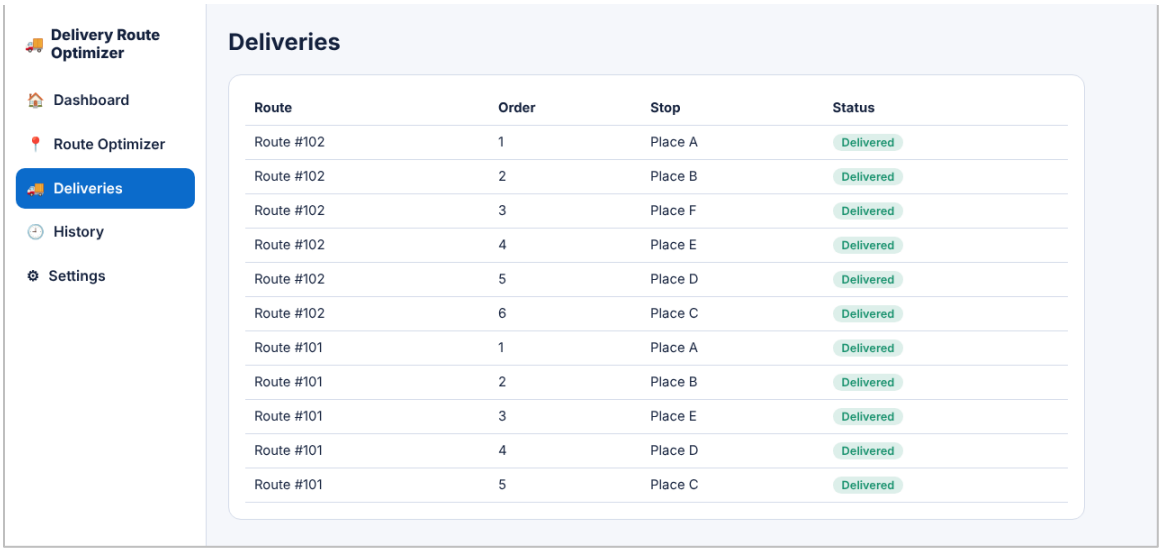


Figure 7.9: Deliveries page

Each stop of today's routes is listed with its order and its status (Delivered, On the way or Pending).

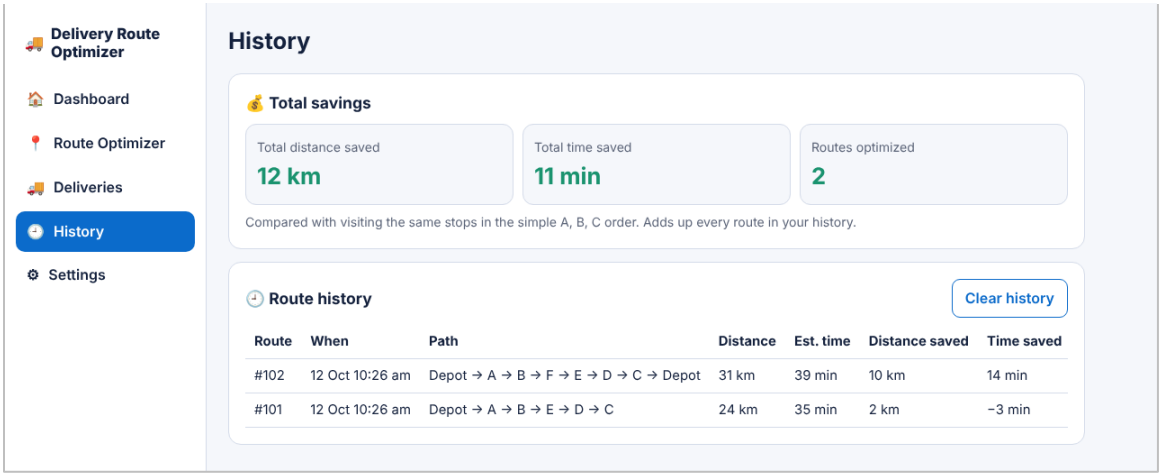


Figure 7.10: History page

The page shows the total distance and time saved, and every route with its path, distance, estimated time and savings.

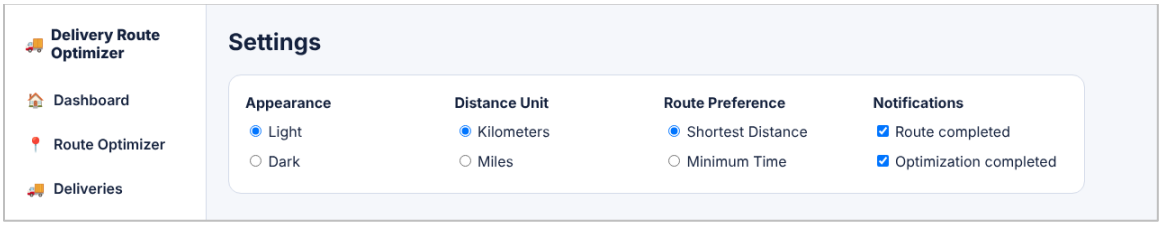


Figure 7.11: Settings page

The user can choose light or dark theme, kilometres or miles, shortest distance or minimum time, and which notifications to show.

8. CONCLUSION & FUTURE SCOPE

The Delivery Route Optimizer meets the objectives set for it. The delivery network is stored as a weighted graph in an adjacency list, Dijkstra's algorithm with a min-heap computes the shortest path from the depot to every delivery point, and the nearest-remaining-stop rule gives an optimized delivery sequence. On the sample network the optimized sequence Depot → A → B → E → D → C is 24 km, against 26 km for the naive A, B, C order, a saving of 2 km (8%). With an extra stop F added and a return to the depot, the saving is 10 km (31 km against 41 km).

The Java console program and the web application give the same result on the sample network, and the web application adds a clickable map, an animated delivery, a deliveries list, a history of routes and settings for units and route preference. The nearest-stop rule is a greedy heuristic, so it gives a good route quickly but not always the best possible tour; the shortest path between any two places, however, is always exact because it comes from Dijkstra's algorithm.

Future Scope

The project can be extended in stages:

- **Near term:** let users save, load and export their own road networks and routes, and compare the nearest-stop result with the exact best order for small numbers of stops.
- **Medium term:** incorporate real-time traffic data for dynamic route re-computation, integrate GPS and map APIs for live geolocation and turn-by-turn navigation, and extend the system to the Vehicle Routing Problem (VRP) for several delivery vehicles.
- **Long term:** apply heuristic and approximation algorithms such as genetic algorithms for large-scale multi-stop optimisation, and explore machine-learning-based delivery time prediction.

9. REFERENCES

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959. https://doi.org/10.1007/BF01386390

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[3] “Dijkstra’s algorithm,” Wikipedia. https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm

[4] “Bellman–Ford algorithm,” Wikipedia. https://en.wikipedia.org/wiki/Bellman%E2%80%93Ford_algorithm

[5] “A* search algorithm,” Wikipedia. https://en.wikipedia.org/wiki/A*_search_algorithm

[6] “Travelling salesman problem,” Wikipedia. https://en.wikipedia.org/wiki/Travelling_salesman_problem

[7] “Vehicle routing problem,” Wikipedia. https://en.wikipedia.org/wiki/Vehicle_routing_problem

[8] Google OR-Tools, “Routing.” https://developers.google.com/optimization/routing

[9] Project OSRM, “Open Source Routing Machine.” https://project-osrm.org/

[10] GraphHopper, “GraphHopper Routing Engine.” https://www.graphhopper.com/